

Linux 系统管理

Linux 系统管理是运维工作的基础，包括系统监控、进程管理、文件系统、用户权限、日志和网络配置等核心技能。

系统监控

系统监控帮助了解服务器运行状态，及时发现性能瓶颈和异常。

top/htop：实时进程监控

top：系统自带

top

常用快捷键
P：按 CPU 使用率排序
M：按内存使用率排序
l：显示所有 CPU 核心
q：退出

htop：增强版（需安装）

htop

优势：
- 彩色显示
- 支持鼠标操作
- 树状视图
- 更直观的界面

top 输出解读：

字段	含义	说明
PID	进程 ID	唯一标识
USER	用户	进程所有者
PR	优先级	数值越小优先级越高
NI	Nice 值	-20 到 19，影响优先级
VIRT	虚拟内存	进程使用的虚拟内存
RES	resident 内存	实际使用的物理内存
SHR	共享内存	可与其他进程共享的内存
S	状态	R 运行/S 睡眠/Z 僵尸
%CPU	CPU 使用率	百分比
%MEM	内存使用率	百分比
TIME+	CPU 时间	进程占用的 CPU 时间
COMMAND	命令	进程名称

TIP

生产环境推荐使用 htop，界面更友好，功能更强大。

vmstat: 虚拟内存统计

每2秒刷新一次，共5次

```
vmstat 2 5
```

```
# 输出示例:
# procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
# r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
# 1  0       0 123456 12345 678901   0   0    10    20  100  200 5  2 93  0  0
```

关键指标:

列	含义	正常范围
r	运行队列长度	< CPU 核心数
b	阻塞进程数	0
swpd	交换分区使用	越低越好
free	空闲内存	越高越好
si/so	swap in/out	应为 0
bi/bo	块设备 IO	根据负载
us	用户 CPU	< 70%
sy	系统 CPU	< 30%
id	空闲 CPU	越高越好
wa	IO 等待	< 10%

CAUTION

如果 si/so 不为 0，说明内存不足，正在使用 swap，应增加内存或优化应用。

iostat: 磁盘 IO 监控

安装 sysstat 包

```
sudo apt install sysstat # Debian/Ubuntu
sudo yum install sysstat # CentOS/RHEL
```

每2秒刷新一次

```
iostat -x 2
```

```
# 输出示例:
# Device rrqm/s wrqm/s  r/s   w/s  rkB/s  wkB/s avgrq-sz avgqu-sz   await  svctm
%util
# sda      0.00   1.00  0.50  2.00   10    20    20.00    0.01    2.00  1.00   0.25
```

关键指标:

- %util: 磁盘利用率，接近 100% 表示磁盘饱和
- await: IO 请求平均等待时间，过高表示磁盘瓶颈
- svctm: IO 请求平均服务时间
- avgqu-sz: 平均队列长度

TIP

%util 高但 await 低，说明磁盘繁忙但性能好；两者都高说明磁盘成为瓶颈。

sar: 系统活动报告

查看历史数据

```
sar -u          # CPU 使用率
sar -r          # 内存使用
sar -b          # IO 传输
sar -n DEV      # 网络统计
sar -q          # 队列长度和负载
```

查看特定时间段

```
sar -u -f /var/log/sa/sa15 # 查看15日的数据
```

实时监控

```
sar -u 1 5 # 每1秒采样，共5次
```

sar 优势:

- 可以查看历史数据（默认保存 7 天）
- 数据存储在 /var/log/sa/
- 支持多种指标（CPU、内存、IO、网络）

进程管理

ps: 进程快照

查看所有进程

```
ps aux
```

输出字段:

```
# USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
```

查找特定进程

```
ps aux | grep nginx
ps -ef | grep java
```

按条件筛选

```
ps aux --sort=-%cpu | head -10 # CPU 使用率前10
ps aux --sort=-%mem | head -10 # 内存使用率前10
```

查看进程树

```
ps auxf # 树状显示
pstree # 更清晰的树状图
```

进程状态:

状态	含义
----	----

R	运行中
S	可中断睡眠
D	不可中断睡眠
Z	僵尸进程
T	停止/追踪

kill: 终止进程

```
# 发送信号

kill 1234          # 默认 SIGTERM (优雅终止)
kill -9 1234       # SIGKILL (强制终止)
kill -15 1234      # SIGTERM (显式指定)
kill -1 1234       # SIGHUP (重新加载配置)
```

```
# 批量终止

killall nginx      # 按名称终止
pkill -f "java.*app" # 按模式匹配
```

```
# 查看信号列表
```

```
kill -l
```

常用信号:

- SIGTERM (15): 请求终止, 允许清理资源
- SIGKILL (9): 强制终止, 无法捕获
- SIGHUP (1): 重新加载配置
- SIGINT (2): 中断 (Ctrl+C)
- SIGSTOP (19): 暂停
- SIGCONT (18): 继续

CAUTION

优先使用 SIGTERM, 给进程清理资源的机会。SIGKILL 是最后手段。

nice/renice: 调整优先级

```
# nice: 启动时设置优先级

nice -n 10 ./slow_script.sh # 较低优先级
nice -n -5 ./important_task # 较高优先级 (需要 root)

# renice: 修改运行中进程的优先级

renice -n 10 -p 1234 # 降低优先级
renice -n -5 -p 1234 # 提高优先级 (需要 root)

# 优先级范围: -20 (最高) 到 19 (最低)
```

NOTE

Nice 值越低, 优先级越高。普通用户只能降低优先级 (增大 nice 值)。

systemd: 现代 init 系统

服务管理

```
systemctl start nginx      # 启动服务
systemctl stop nginx       # 停止服务
systemctl restart nginx    # 重启服务
systemctl reload nginx     # 重载配置
systemctl status nginx     # 查看状态
```

开机自启

```
systemctl enable nginx     # 启用开机自启
systemctl disable nginx    # 禁用开机自启
```

查看日志

```
journalctl -u nginx        # 查看服务日志
journalctl -u nginx -f      # 实时跟踪
journalctl -u nginx --since "1 hour ago"
```

查看所有服务

```
systemctl list-units --type=service
systemctl list-unit-files    # 查看所有单元文件
```

systemd 优势:

- 并行启动, 加快开机速度
- 依赖管理, 自动处理服务顺序
- 统一的日志管理 (journalctl)
- 强大的资源配置能力

文件系统管理

df: 磁盘空间使用情况

查看所有挂载点

```
df -h    # 人类可读格式
```

输出示例:

# Filesystem	Size	Used	Avail	Use%	Mounted on
# /dev/sda1	50G	20G	30G	40%	/
# tmpfs	7.8G	0	7.8G	0%	/dev/shm

查看特定类型

```
df -hT      # 显示文件系统类型
df -h /home  # 查看特定目录
```

TIP

定期监控磁盘使用率, 建议在达到 80% 时发出警告, 90% 时紧急处理。

du: 目录大小统计

查看目录大小

```
du -sh /var/log      # 单个目录总大小
```

```
du -sh /var/*        # 子目录大小
```

查找大文件

```
du -ah /var | sort -rh | head -20 # 最大的20个文件/目录
```

深度控制

```
du -h --max-depth=1 /home # 只显示第一层
```

fdisk: 分区管理

查看分区表

```
sudo fdisk -l
```

交互式分区

```
sudo fdisk /dev/sdb
```

常用命令:

p: 打印分区表

n: 新建分区

d: 删除分区

t: 修改分区类型

w: 写入并退出

q: 退出不保存

CAUTION

分区操作有风险，务必先备份重要数据。生产环境建议使用 LVM。

LVM: 逻辑卷管理

LVM (Logical Volume Manager) 提供灵活的存储管理。

创建物理卷

```
pvccreate /dev/sdb1
```

创建卷组

```
vgcreate vg_data /dev/sdb1
```

创建逻辑卷

```
lvcreate -L 100G -n lv_home vg_data
```

格式化并挂载

```
mkfs.ext4 /dev/vg_data/lv_home
```

```
mount /dev/vg_data/lv_home /home
```

扩展逻辑卷 (在线)

```
lvextend -L +50G /dev/vg_data/lv_home
resize2fs /dev/vg_data/lv_home # ext4
xfs_growfs /home # xfs
```

查看信息

```
pvdisplay
vgdisplay
lvdisplay
```

LVM 优势:

- 在线扩展/缩小卷
- 快照功能 (备份)
- 跨多个磁盘聚合存储
- 灵活的存储池管理

用户与权限

用户管理

创建用户

```
sudo useradd -m -s /bin/bash username
sudo passwd username
```

删除用户

```
sudo userdel -r username # -r 同时删除家目录
```

修改用户

```
sudo usermod -aG sudo username # 添加到 sudo 组
sudo usermod -s /sbin/nologin username # 禁止登录
```

查看用户

```
id username
whoami
cat /etc/passwd
```

TIP

-m 创建家目录, -s 指定 shell, -aG 追加到组 (不覆盖原有组)。

文件权限

查看权限

```
ls -l file.txt
# -rw-r--r-- 1 user group 1024 Jan 1 12:00 file.txt
```

权限分解:

```
# -      rw-      r--      r--
# |      |      |      |
# 类型   所有者   所属组   其他用户
```

修改权限（符号模式）

```
chmod u+x script.sh      # 所有者添加执行权限
chmod g-w file.txt       # 组移除写入权限
chmod o=r file.txt       # 其他用户只读
chmod a+x script.sh      # 所有用户添加执行权限
```

修改权限（数字模式）

```
chmod 755 script.sh      # rwxr-xr-x
chmod 644 file.txt       # rw-r--r--
chmod 600 private.key    # rw-----
```

数字权限对照：

权限	数字	含义
r	4	读
w	2	写
x	1	执行
rwx	7	4+2+1
rw-	6	4+2
r-x	5	4+1

chown：修改所有者

修改所有者

```
sudo chown newuser file.txt
sudo chown newuser:newgroup file.txt  # 同时修改组和所有者
```

递归修改

```
sudo chown -R user:group /directory/
```

sudo：提权执行

使用 sudo

```
sudo command
```

编辑 sudoers 文件（必须用 visudo）

```
sudo visudo
```

添加规则

```
username ALL=(ALL:ALL) ALL
%admin ALL=(ALL) NOPASSWD: ALL  # admin 组无需密码
```

查看 sudo 权限

```
sudo -l
```


CAUTION

始终使用 `visudo` 编辑 `sudoers` 文件，它会检查语法错误，避免锁定系统。

日志管理

journalctl: systemd 日志

查看所有日志

```
journalctl
```

查看特定服务

```
journalctl -u nginx
```

```
journalctl -u sshd
```

时间过滤

```
journalctl --since "2024-01-01"
```

```
journalctl --since "1 hour ago"
```

```
journalctl --until "10 min ago"
```

优先级过滤

```
journalctl -p err          # 错误及以上
```

```
journalctl -p warning      # 警告及以上
```

实时跟踪

```
journalctl -f
```

```
journalctl -u nginx -f
```

查看内核日志

```
journalctl -k
```

TIP

`journalctl` 是 `systemd` 系统的统一日志接口，取代了传统的 `syslog`。

logrotate: 日志轮转

配置文件

```
/etc/logrotate.conf          # 主配置
```

```
/etc/logrotate.d/            # 子配置目录
```

示例配置 (/etc/logrotate.d/nginx)

```
/var/log/nginx/*.log {
    daily                # 每天轮转
    missingok            # 日志不存在不报错
    rotate 14            # 保留14个备份
    compress             # 压缩旧日志
    delaycompress        # 延迟一天压缩
    notifempty           # 空文件不轮转
    create 0640 www-data adm
```

```

sharedscripts
postrotate
    [ -f /var/run/nginx.pid ] && kill -USR1 `cat /var/run/nginx.pid`
endscript
}

```

手动测试

```

logrotate -d /etc/logrotate.d/nginx # 调试模式
logrotate -f /etc/logrotate.d/nginx # 强制执行

```

常用选项:

- daily/weekly/monthly: 轮转频率
- rotate N: 保留 N 个备份
- size N: 达到指定大小后轮转
- compress: 压缩旧日志
- copytruncate: 复制后清空 (适合不支持 reopen 的应用)

rsyslog: 传统日志系统

配置文件

```

/etc/rsyslog.conf
/etc/rsyslog.d/

```

基本规则

```

# 格式: facility.priority    action
*.info;mail.none;authpriv.none;cron.none    /var/log/messages
authpriv.*                                    /var/log/secure
mail.*                                         /var/log/maillog
cron.*                                         /var/log/cron

```

远程日志

```

*. * @remote-host:514    # TCP
*. * @remote-host:514    # UDP

```

重启服务

```

sudo systemctl restart rsyslog

```

NOTE

现代 Linux 发行版通常同时运行 rsyslog 和 journald, journald 作为主要日志系统。

网络配置

ip: 网络接口管理

查看网络接口

```

ip addr show
ip -brief addr show    # 简洁输出

```

查看路由表

```
ip route show
ip route get 8.8.8.8      # 查询特定IP的路由
```

```
# 查看邻居表 (ARP)
```

```
ip neigh show
```

```
# 启用/禁用接口
```

```
sudo ip link set eth0 up
sudo ip link set eth0 down
```

```
# 添加IP地址
```

```
sudo ip addr add 192.168.1.100/24 dev eth0
sudo ip addr del 192.168.1.100/24 dev eth0
```

TIP

ip 命令取代了旧的 ifconfig、route、arp 等命令，功能更强大。

ss: Socket 统计

```
# 查看所有连接
```

```
ss -tulpn
```

```
# 输出示例:
```

```
# Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
# tcp    LISTEN  0      128   0.0.0.0:22          0.0.0.0:*      users:(...)
# tcp    ESTAB   0        0   192.168.1.10:22     192.168.1.5:54321
```

```
# 过滤
```

```
ss -t state established  # 已建立的TCP连接
ss -t state listening    # 监听端口
ss -u                    # UDP连接
ss -x                    # Unix socket
```

```
# 统计
```

```
ss -s                    # 摘要统计
```

ss vs netstat:

- ss 更快，直接从内核获取信息
- netstat 已废弃，但仍在广泛使用
- 推荐使用 ss

iptables: 防火墙

```
# 查看规则
```

```
sudo iptables -L -n -v
sudo iptables -L INPUT -n -v
```

```
# 基本规则
```

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT    # 允许SSH
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT    # 允许HTTP
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT    # 允许HTTPS
sudo iptables -A INPUT -j DROP                        # 拒绝其他
```

删除规则

```
sudo iptables -D INPUT 3    # 删除第3条规则
```

保存规则 (Debian/Ubuntu)

```
sudo iptables-save > /etc/iptables/rules.v4
```

保存规则 (CentOS/RHEL)

```
sudo service iptables save
```

CAUTION

iptables 规则复杂且容易出错，生产环境建议使用 firewalld 或 ufw 简化管理。

NetworkManager: 网络管理

使用 nmcli

```
nmcli device status    # 查看设备状态
nmcli connection show  # 查看连接
nmcli device wifi list  # 扫描WiFi
```

配置静态IP

```
sudo nmcli con mod "eth0" ipv4.addresses 192.168.1.100/24
sudo nmcli con mod "eth0" ipv4.gateway 192.168.1.1
sudo nmcli con mod "eth0" ipv4.dns "8.8.8.8 8.8.4.4"
sudo nmcli con mod "eth0" ipv4.method manual
sudo nmcli con up "eth0"
```

使用 nmtui (文本界面)

```
sudo nmtui
```

本章完

Shell 脚本自动化

服务管理与高可用